

De Gouden Tarwe SQL Database Challenge

(Day 3)

Python - MySQL

Problem Statement

De Gouden Tarwe (Dutch for The Golden Wheat) is a premium bakery chain based in the Netherlands. Established in 2018, the company has grown rapidly and now operates more than 30 branches across 20 major Dutch cities including Amsterdam, Rotterdam, Utrecht, The Hague, Eindhoven, and Groningen.

The bakery sells authentic Dutch baked goods from everyday bread like tijgerbrood and volkorenbrood, to traditional pastries like stroopwafels, appeltaart, bossche bollen, and seasonal specialties like oliebollen (only sold in December) and pepernoten (only sold during the Sinterklaas season in November).

The company employs over 450 people across all branches: bakers, assistant bakers, sales staff, branch managers, delivery drivers, and support staff. It serves tens of thousands of customers, many of whom participate in a loyalty program with tiered benefits (Bronze, Silver, Gold, Platinum).

The Current Problem

De Gouden Tarwe currently stores all its operational data in flat CSV files. As the business has grown, this approach has become unmanageable:

1. Data redundancy: The same product name, price, and description are repeated in every transaction record.
2. No formal relationships: There is no way to enforce that a branch referenced in an order actually exists.
3. No data integrity: A typo in a city name creates a new city that does not actually exist.
4. Impossible to answer business questions: Simple queries like What is our best-selling product in Rotterdam? require manually cross-referencing multiple files.
5. Performance problems: With millions of rows, opening and searching CSV files is painfully slow.

Progress – Day 3

On Day 3, we loaded the CSV dataset into the database created previously. Prior to loading, we cleaned, reformatted, and validated the data to ensure it matched our schema. If constraints such as unique values or column requirements were set during database creation, invalid data would cause errors during loading. Transforming the data in advance helps reduce these errors.

On Day 3, I introduced Python to handle both data transformation and loading. After confirming the data was clean, I loaded it into MySQL using the SQLAlchemy library. Below are syntax examples and potential errors that may arise during these processes.

Library we used

```
import pandas as pd
from sqlalchemy import create_engine
from tqdm import tqdm
```

```
import math
```

Tqdm and math are optional, they are used to create a progress bar during the data loading process.

Create engine

```
engine =  
create_engine('mysql+pymysql://username:password@localhost:3306/database_  
name')
```

Change to date and time format

```
df['opened_date'] = pd.to_datetime(df['opened_date'])  
df['closing_time'] = pd.to_datetime(df['closing_time'], format='%Y-%d-  
%m')  
df['production_start_time'] =  
pd.to_datetime(df['production_start_time']).dt.time
```

Change 0 to null

```
df['order_id'] = df['order_id'].replace(0, None)
```

Check data duplicate

```
df.duplicated().sum()
```

Load data

```
df.to_sql('table name', engine, if_exists='append', index=False)
```

Because of the large data volume, the device cannot load data directly into the database. To address this, try increasing the database timeout, temporarily disabling table constraints such as foreign keys and unique checks, turning off autocommit, and increasing the Python chunk size during loading.

Turn off checks in MySQL

```
SET FOREIGN_KEY_CHECKS = 0;  
SET UNIQUE_CHECKS = 0;  
SET AUTOCOMMIT = 0;
```

Disabling database checks means that the database does not need to enforce strict validation rules. Data validation takes a long time. On some devices, it can cause overheating and give slow performance. To re enable it, simply change 0 to 1 after the data loading process is complete.

Add timeout in MySQL

```
SET GLOBAL max_allowed_packet = 536870912;  
SET GLOBAL net_read_timeout = 600;  
SET GLOBAL net_write_timeout = 600;
```

These configurations are used to optimize the database connection when handling large data transfers. Increasing the `max_allowed_packet` value allows the server to process larger queries or data inserts, while extending `net_read_timeout` and `net_write_timeout` gives the server more time to receive and send data without terminating the connection prematurely. Together, these settings help prevent common issues such as packet size errors or connection timeouts during large data imports or ETL processes.

Load data using chunk size and tqdm

```
chunk_size = 1000
total_rows = len(df)
total_chunks = math.ceil(total_rows / chunk_size)

for i in tqdm(range(total_chunks)):
    start = i * chunk_size
    end = start + chunk_size
    chunk = df.iloc[start:end]

    chunk.to_sql(
        'table_name',
        engine,
        if_exists='append',
        index=False
    )
```

Chunk size is used to divide a large dataset into smaller batches during data processing or database insertion. By sending data in smaller portions rather than all at once, the loading process becomes more efficient and stable. This method improves performance, reduces memory usage, and lowers the risk of errors such as connection timeouts or packet size limitations when handling large volumes of data.

Please note that if an issue or failure during the loading process, the data entered into the database must be deleted first.

```
TRUNCATE TABLE nama_tabel;
```

There's one more issue that often comes up: differences in column order between datasets and database tables. To resolve this, you can actually make adjustments during the data load process, you don't necessarily have to change the column order in the database. Here are two ways to handle this:

Make additional adjustments when loading data, the order matches the data in the datasets

```
db_columns = ['first_column', 'second_column', 'third_column']
df = df[db_columns]
df.to_sql('employees', engine, if_exists='append', index=False)
```

Reorder the columns in the database

```
ALTER TABLE table_name;
MODIFY COLUMN column_change in data type of column AFTER column_change;
```

This is the amount of data we were able to load. We successfully loaded 72 million records from 24 columns.

	AZ tables	123 num_of_rows
1	branch_operating_hours	245
2	branches	35
3	cities	20
4	customer_feedback	50,000
5	customers	200,000
6	daily_production	1,037,791
7	employee_shifts	26,550
8	employees	496
9	equipment	748
10	ingredients	45
11	inventory_stock	1,575
12	loyalty_transactions	200,000

13	order_items	50,198,970
14	orders	20,004,236
15	price_history	390
16	product_categories	26
17	product_recipes	406
18	products	78
19	promotions	50
20	purchase_order_items	90,666
21	purchase_orders	18,173
22	regions	10
23	suppliers	12
24	waste_log	320,420
25	TOTAL ALL TABLES	72,150,942